NODIRBEK MAKHTUMOV

AI/ML muhandisi portfoliosi

Computer Vision · Edge AI · Modellarni optimallashtirish

E-7 (Special Ability) vizasi mavjud · Koreyada ishlash huquqi bor

stark.developer2877@gmail.com · +82 10-5751-0750
github.com/Nodir0705 · linkedin.com/in/nodirbek-makhtumov-03084a194
nodir0705.github.io
Daejeon, Janubiy Koreya · 2026-yil noyabr

Mundarija

Ish joyidagi loyihalar

01 **Aqlli chiqindi mashinalari boshqaruv tizimi** 2023.04 – hozirgacha 2

02 **Edge-Aware segmentatsiya pipeline'i** 2023.09 – hozirgacha 4

03 **SAM2 asosida video avto-annotatsiya vositasi** 2024 6

Shartnoma loyihalari

04 **Aqlli yuk mashinalari parkovkasi va ID aniqlash** 2025.09 – 2025.11 7

05 **Yuzni tanib davomatni qayd etuvchi kiosk** 2026.07 10

06 **Sales Agent (email orqali narx taklifi, inson tasdig'i bilan)** 2026.10 12

Shaxsiy loyihalar

07 **Rasm asosidagi klassifikator barqarorligi tadqiqoti** 2026 14

08 **IoT xavfsizlik shlyuzi (Orange Pi 5 · RK3588S)** 2026 16

09 **Smart Pointer (nutqqa qarab slayd qismini ajratib ko'rsatish)** 2026 17

Texnik ko'nikmalar · Ta'lim · Tillar · Viza 19

Qisqacha ma'lumot

Edge qurilmalarda ishlaydigan kompyuter ko'rish tizimlarini quraman.

- Obyekt aniqlash, segmentatsiya va real vaqtli video tahlilga asoslangan production vision tizimlarini **3 yildan ortiq** ishlab chiqib, tijorat maqsadida joriy etdim.
- Model dizayni va o'qitishdan tortib **TensorRT/ONNX kvantlash va pruning**, NVIDIA Jetson, Hailo-8 va RK3588 NPU'ga deploymentgacha butun jarayonni o'zim bajaraman.
- FastAPI inferens servisi, PostgreSQL'ga metama'lumot yozish va Docker asosidagi edge deployment bilan tizimga **boshdan-oxir** javob beraman.
- Modelni bitta aniqlik raqami bilan cheklanmay, **sinflar bo'yicha metrikalar, adversarial barqarorlik va qurilmadagi benchmarklar** bilan tekshiraman.
- **Inson tasdig'i bilan ishlaydigan LLM agentlar** ham quraman va ularni eval to'plamlarida o'lchayman.
- O'zbek (ona tili), ingliz, koreys va rus tillarini bilaman. **E-7 (Special Ability) vizasi** mavjud, Koreyada darhol ish boshlay olaman.

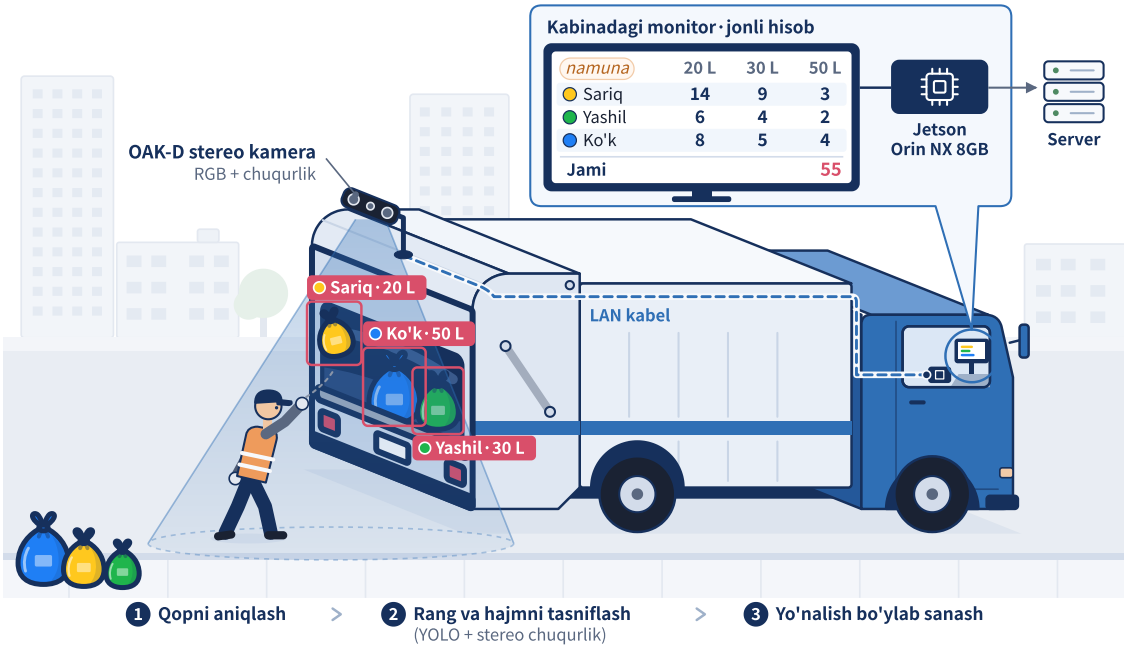
30%	25+ FPS	8.6×	76% → 92%
GPU xotira sarfi kamaydi	4 ta oqim parallel	NPU inferens tezlashuvi	Truck-ID OCR aniqligi
YOLOv8 backbone almashuvi	Real vaqtli segmentatsiya	Hailo-8 · jitter 95× kam	Deskew moduli qo'shildi

01 Aqlli chiqindi mashinalari boshqaruv tizimi

YOLO bilan chiqindi tasnifi va yuk hajmini baholash — bortda edge inferens

Netvision Telecom · AI/ML muhandisi · 2023.04 – hozirgacha

YOLOv8 | ResNet50 | TensorRT | Jetson Orin NX | DLA | Docker



Biznes ta'siri

Bo'shagan xotira hisobiga mashinadagi bitta Jetson platasida vision model va mavjud boshqaruv jarayonlari birga, uzluksiz ishlay boshladi. Asosiy natija — yangi plata qo'shmasdan, mavjud qurilmaning o'zida vision funksiyasi qo'shildi.

Muammo

Jetson Orin NX'da chiqindi turini tasniflash va yuk hajmini baholashni real vaqtda bajarish kerak edi. Biroq standart YOLOv8 modeli GPU xotirasini juda ko'p egallagani uchun uni shu platadagi boshqa boshqaruv jarayonlari bilan birga doimiy ishlatish qiyin edi.

Texnik yondashuv

- **Backbone'ni almashtirish** — YOLOv8 standart backbone'ini ResNet50 ga almashtirib, mavjud PANet neck'ning feature map o'lchamlariga mos 1x1 convolution kanal adapterlarini qo'shdim.
- **DLA offloading** — ResNet qatlamlari Jetson DLA (Deep Learning Accelerator)da qo'llab-quvvatlangani uchun hisoblashlarni GPU'dan DLA'ga taqsimladim. Xotira asosan shu hisobiga tejaldi.
- **Nazoratli A/B baholash** — Bir xil maqsadli qurilmada mAP, FPS va xotira sarfini avvalgi va yangi arxitektura o'rtasida bevosita taqqosladim.
- **Optimallashtirish va deployment** — Kvantlash va TensorRT konvertatsiyasi bilan inferens vaqtini yana qisqartirdim, issiqlik va quvvat cheklovlari doirasida Docker konteyneriga joylab, takrorlanadigan deployment qildim.

Aqlli chiqindi mashinalari boshqaruv tizimi

Arxitekturani taqqoslash

Avvalgi arxitektura

YOLOv8m

Backbone — CSPDarknet53

Stem + Conv Layers
3 → 64ch

C2f Blocks (Stage 2–3)
64 → 192ch

C2f Blocks (Stage 4)
192 → 384ch

C2f + SPPF (Stage 5)
384 → 576ch

Neck — PANet / FPN bir xil

Upsample + Concat + C2f
Ko'p masshtabli birlashtirish

Head — Decoupled bir xil

Detect (P3, P4, P5)
cls + bbox

Yangilangan arxitektura

YOLOv8 + ResNet50

Backbone — ResNet50 (almashtirildi)

Conv7x7 + MaxPool
3 → 64ch

Layer 1–2 (Bottleneck)
64 → 512ch

Layer 3 (Bottleneck)
512 → 1024ch

Layer 4 (Bottleneck)
1024 → 2048ch

1x1 Conv kanal adapteri — qo'shildi

1x1 Conv kanal adapteri
512/1024/2048 → 192/384/576 · neck o'lchamiga moslash

DLA offloading — asosiy optimallashtirish

DLA offloading
ResNet qatlamlari (Conv · BN · ReLU) DLA'da ishlaydi → GPU'dan ajratildi

CSPDarknet'ning C2f bloklarida maxsus amallar bor va DLA ularni to'liq qo'llab-quvvatlamaydi, shuning uchun ular faqat GPU'da ishlaydi.

Neck — PANet / FPN bir xil

Upsample + Concat + C2f
Ko'p masshtabli birlashtirish

Head — Decoupled bir xil

Detect (P3, P4, P5)
cls + bbox

Backbone	Asosiy o'zgarish	Tejash sababi	GPU xotirasi
CSPDarknet → ResNet50	1x1 Conv adapter	ResNet qatlamlari → DLA	↓ 30%

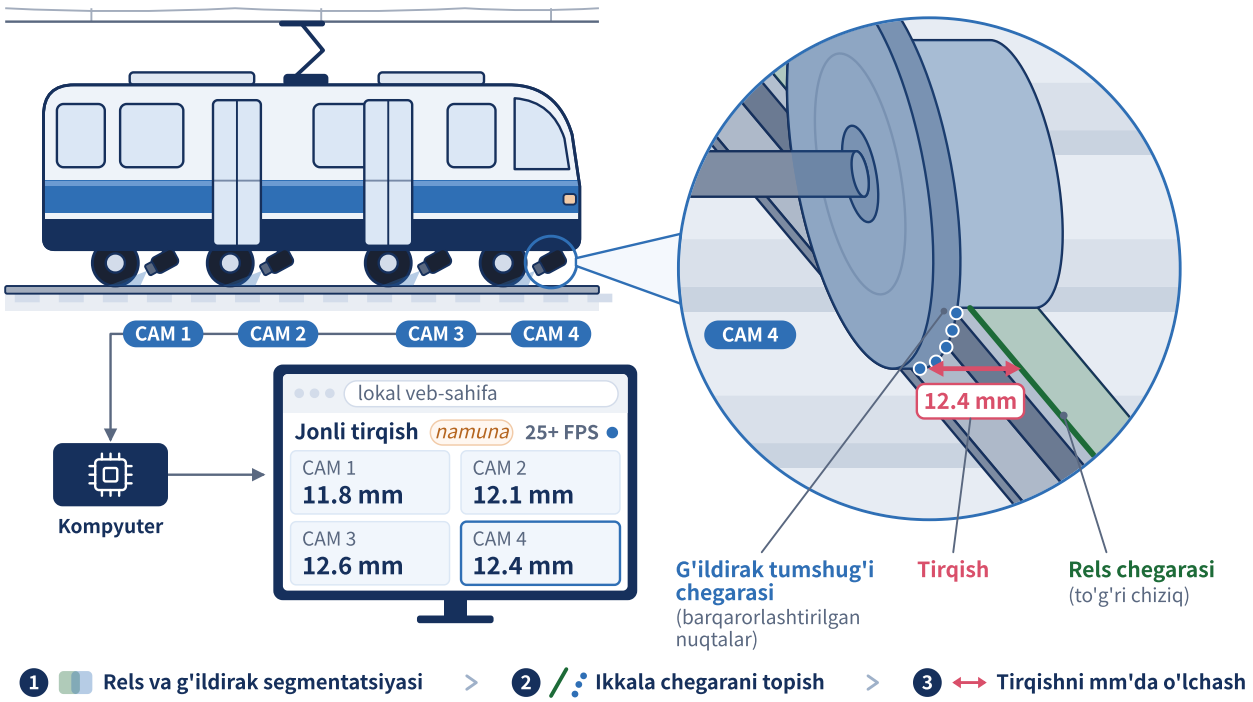
02

Edge-Aware segmentatsiya pipeline'i

Rels–g'ildirak tirqishini real vaqtda o'lchash tizimi — 4 ta oqim parallel

Netvision Telecom · AI/ML muhandisi · 2023.09 – hozirgacha

PyTorch | RANSAC | FastAPI | PostgreSQL | OpenCV | Multi-threading



Biznes ta'siri

Odamlar qo'lda bajaradigan rels–g'ildirak tirqishi tekshiruvini ko'p kamerali avtomatik o'lchash bilan almashtirdim. Barcha o'lchovlar vaqt belgisi (timestamp) bilan saqlanadi, shu sababli tarixni kuzatish va keyinchalik tahlil qilish mumkin.

Muammo

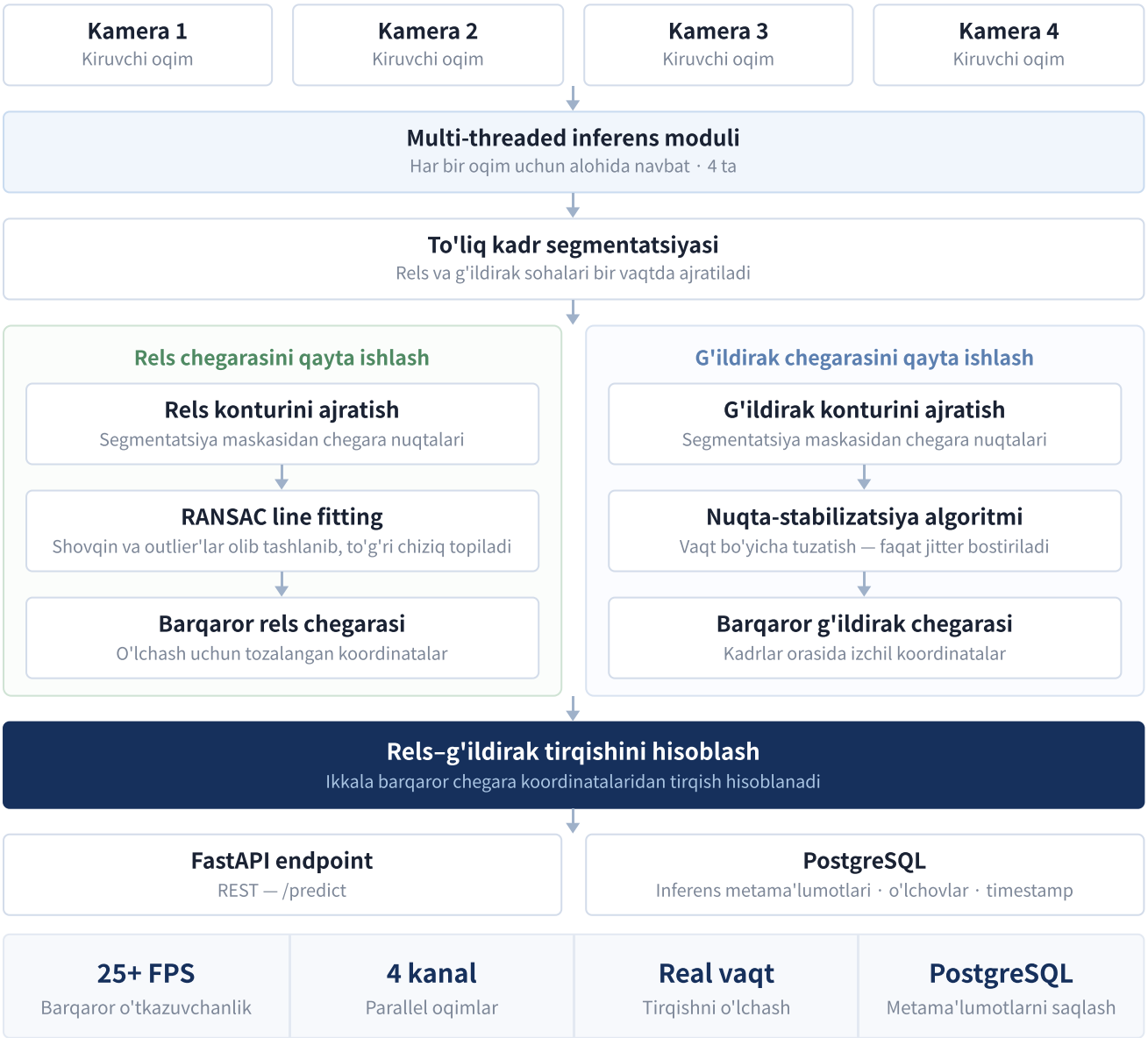
Bir nechta kamera oqimida rels va g'ildirak chegaralarini real vaqtda aniq topish kerak edi. Oddiy segmentatsiya modellari, ayniqsa egri chegaralarda, shovqinli va har kadrda tebranib turadigan chegaralar berardi, shu sababli ular ustida hisoblangan tirqish o'lchovlariga ishonib bo'lmasdi.

Edge-Aware segmentatsiya pipeline'i

Tizim arxitekturasi

Texnik yondashuv

- **Yengil segmentatsiya modeli** — Inferens tezligini saqlagan holda chegara aniqligini yo'qotmaydigan yengil arxitektura rels va g'ildirak sohalarini bir vaqtda ajratadi.
- **RANSAC asosida chegarani ajratish** — Rels chegarasiga RANSAC regressiyasi bilan geometrik to'g'ri chiziq moslab, segmentatsiya shovqini va outlier nuqtalarni olib tashladim.
- **O'zim yozgan nuqta-stabilizatsiya algoritmi** — G'ildirak chegara nuqtalarini ketma-ket kadrlar bo'ylab barqarorlashtirdim: haqiqiy joy o'zgarishi saqlanadi, faqat kadrlararo jitter bostiriladi.
- **Parallel oqimlarni qayta ishlash** — Har bir oqim uchun alohida navbatga ega multi-threaded pipeline 4 ta kamera oqimini bir vaqtda qayta ishlaydi.
- **Serving va metama'lumotlarni saqlash** — Natijalar FastAPI endpoint orqali beriladi, inferens metama'lumotlari va tirqish o'lchovlari PostgreSQL'ga tuzilgan holda yoziladi.



25+ FPS	4 kanal	Real vaqt	PostgreSQL
Barqaror o'tkazuvchanlik	Parallel oqimlar	Tirqishni o'lchash	Metama'lumotlarni saqlash

03

SAM2 asosida video avto-annotatsiya vositasi

Temporal propagation asosidagi avtomatik annotatsiya pipeline'i

Netvision Telecom · AI/ML muhandisi · 2024

SAM2 | PyTorch | Video Processing | YOLO/COCO Export | OpenCV

> **Biznes ta'siri**
> O'quv dataset tayyorlash jamoaning eng sekin bosqichi edi. Qo'lda annotatsiyani avtomatik tarqatishga almashtirib, yangi vision vazifalarida ma'lumot tayyorlash vaqtini keskin qisqartirdim.

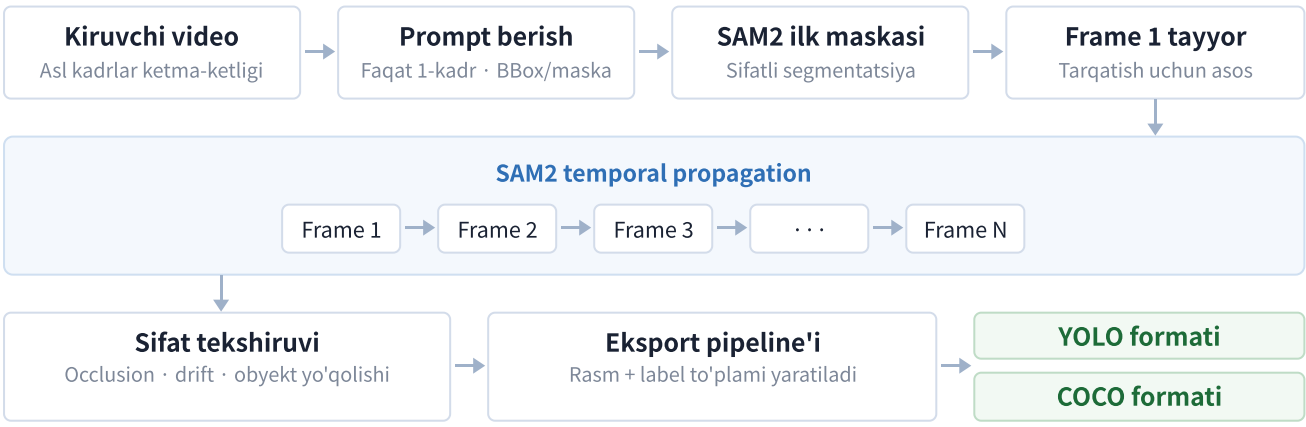
Muammo

Sifatli kompyuter ko'rish modelini o'qitish uchun izchil label'langan katta dataset kerak. Videoni qo'lda annotatsiya qilish juda ko'p vaqt olardi, label mezonlari esa har bir xodimda har xil bo'lardi.

Texnik yondashuv

- **Prompt asosida dastlabki segmentatsiya** — Foydalanuvchi birinchi kadrda bounding box yoki maskani bir marta belgilaydi, SAM2 sifatli dastlabki maskani yaratadi.
- **Temporal propagation** — SAM2 keyingi barcha kadrlarda obyektni kuzatib segmentatsiya qiladi va butun videoga izchil label beradi.
- **Izchillik tekshiruvi** — Occlusion yoki obyekt yo'qolishi sabab buzilgan natijalar ajratiladi.
- **O'quv formatlariga eksport** — YOLO va COCO formatlariga avtomatik o'giradi, natijani mavjud o'qitish freymvorklariga darhol berish mumkin.

Pipeline tuzilmasi



Natijalar

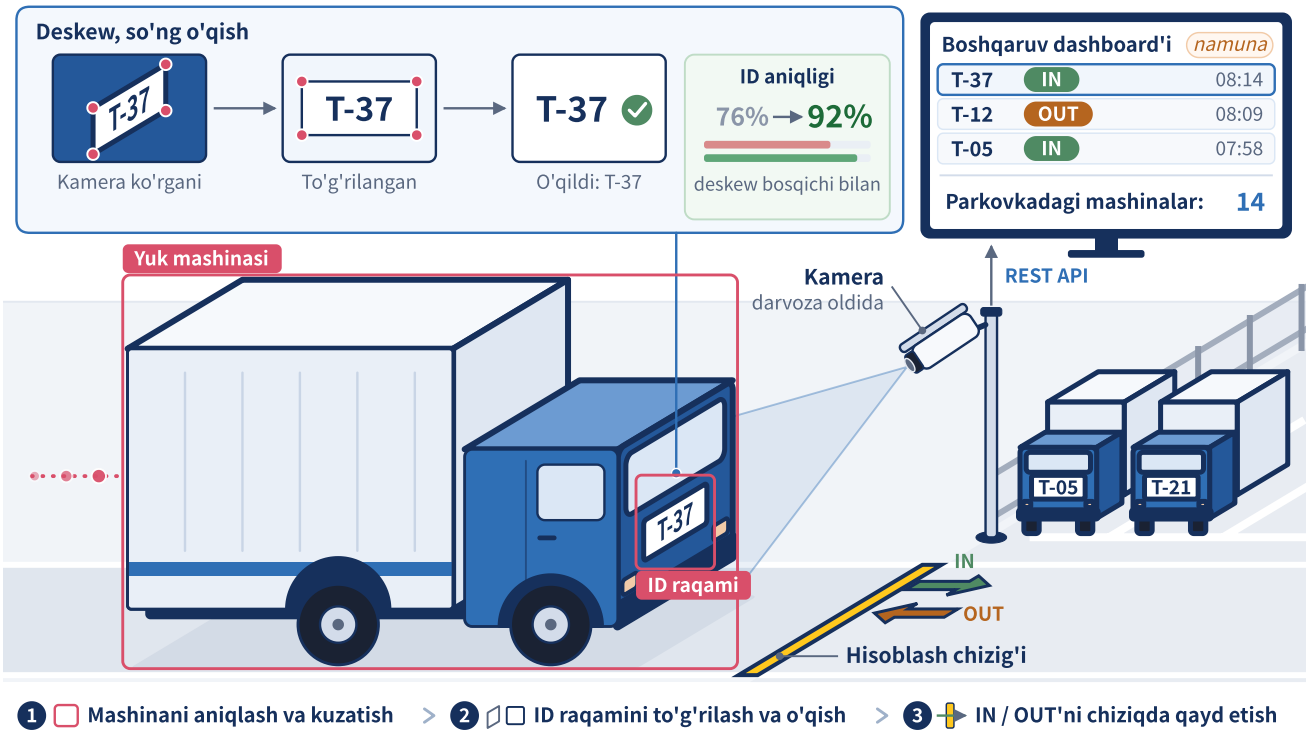
1 kadr	Butun video	10x+	YOLO · COCO
Qo'lda belgilanadi	Avtomatik annotatsiya	Ish tezligi oshdi	Eksport formatlari

04 Aqlli yuk mashinalari parkovkasi va ID aniqlash

Yakka o'zim boshdan-oxir qurgan real vaqtli vision tizimi

Frilans shartnoma loyihasi · AI/ML muhandisi · 2025.09 – 2025.11 ·
github.com/Nodir0705/smart-truck-parking

YOLO | ByteTrack | EasyOCR | OpenCV | Perspective Transform | REST API



Biznes ta'siri

Parkovkaga kirish-chiqishni qog'oz jurnal o'rniga avtomatik qayd etadigan qildim. Bandlik holati boshqaruv ekranida real vaqtda ko'rinadi — operator joyni aylanmay vaziyatni biladi.

Muammo

Yuk mashinalari parkovkasida mashinani aniqlash, noyob ID'ni o'qish va bandlik holatini real vaqtda uzatishni avtomatlashtirish kerak edi. Joyida ID raqamlari qiya burchakdan suratga tushar, yoritish ham yomon edi, shu sabab dastlabki OCR aniqligi atigi 76% edi.

Aqlli yuk mashinalari parkovkasi va ID aniqlash

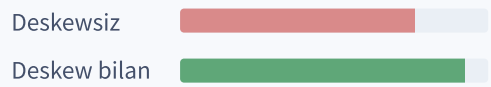
Texnik yondashuv

- **Ko'p bosqichli aniqlash** — YOLO yuk mashinasi joylashuvi va ID raqami ROI'sini alohida topadi; faqat ID sohasi kesib olinib, o'qish bosqichiga uzatiladi.
- **Kuzatish va ID'ni tasdiqlash** — ByteTrack har bir mashina trekini saqlaydi, chiziq kesib o'tilganda IN / OUT hodisasi yaratiladi. ID oxirgi 5 ta o'qish bo'yicha ovoz berish bilan tasdiqlanadi.
- **Deskew pipeline'i** — Contour detection va adaptive threshold bilan ID raqamining 4 burchagini topib, perspective transform va burishni tuzatish orqali buzilishni to'g'rilaydigan modulni o'zim yozdim.
- **EasyOCR integratsiyasi** — Truck ID formatiga moslab sozladim, oldiga kontrastni oshirish va shovqinni tozalash bosqichlarini qo'ydim.
- **Real vaqtli uzatish** — Kirish-chiqish hodisalari va aniqlangan ID'lar REST API orqali backendga real vaqtda yuboriladi va boshqaruv dashboard'ida ko'rsatiladi.
- **Xatolarni tizimli tahlil qilish** — Xato holatlarini turlarga ajratib, asosiy sabab perspektiv buzilish ekanini aniqladim; deskew modulini qo'shishga shu asos bo'ldi.
- **Ochiq kodli versiya** — Deskew modulini qayta yozib, ochiq kodli versiyasini e'lon qildim. Sintetik yaratilgan 200 ta qiyshiq ID raqamida EasyOCR aniqligi 80.5% → 97.5% ga oshdi. Bu mijoz obyektidagi natijadan (76% → 92%) alohida o'lchov.

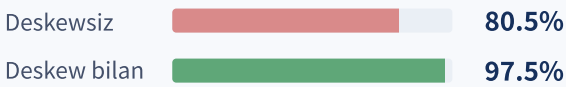
Natijalar

76% → 92%	Real vaqt	Boshdan-oxir	O'zim yozgan
ID aniqligi (mijozda)	Backendga uzatish	Yakka o'zim qurdim	Deskew algoritmi

Mijoz obyekti (topshirilgan tizim)
Joydagi kameralarda ID aniqligi



Ochiq kodli versiya (alohida o'lchov)
200 ta sintetik qiyshiq ID raqami · EasyOCR aniqligi



Xatolarni tizimli tahlil qilib, asosiy sabab perspektiv buzilish ekanini aniqlaganimdan keyin deskew modulini qo'shdim. Ikki natijaning ma'lumotlari va implementatsiyasi farq qiladi, shuning uchun ularni bevosita taqqoslamayman.

Aqlli yuk mashinalari parkovkasi va ID aniqlash

Pipeline tuzilmasi



05

Yuzni tanib davomatni qayd etuvchi kiosk

Uch xil pipeline o'lchab taqqoslangach, Samsung Galaxy Tab'da on-device inferens bilan topshirildi

Frilans shartnoma loyihasi · AI/ML muhandisi · 2026.07 · **github.com/Nodir0705/face-recognition**

SCRFD | ArcFace | InsightFace | Hailo-8 NPU | Raspberry Pi 5 | C++ | Galaxy Tab | TFLite | SQLite

> **Biznes ta'siri**
> Qurilmani taxmin bilan emas, bitta pipeline'ni uch usulda qurib, o'lchov natijalari asosida tanladim. Yakunda Samsung Galaxy Tab'da on-device ishlaydigan davomat kiosk'ini topshirdim: alohida inferens qurilmasi kerak emas, bitta planshetning o'zi yetarli.

Muammo

Real vaqtda yuzni tanib davomatni qayd etuvchi kiosk qurish kerak edi, lekin qaysi qurilmada ishlashi hali hal qilinmagan edi. CPU'dagi inferens sekin edi, ammo kattaroq muammo kechikishning (latency) beqarorligi edi. Foydalanuvchi kamera oldida qancha kutishini oldindan bilolmasa, kiosk sekin tuyuladi.

Texnik yondashuv

- **Qurilma tanlash uchun 3 xil o'lchov** — Bir xil yuzni tanish pipeline'ini uch usulda — Python/CPU, C++ port va Hailo-8 NPU — qurib, o'zim o'lchadim. Python/CPU va Hailo-8 bitta Raspberry Pi 5'da, bir xil rasm bilan taqqoslandi; C++ port esa Raspberry Pi 4'da Python versiyasi bilan taqqoslandi.
- **Model tarkibi** — Yuz SCRFD bilan aniqlanadi, ArcFace 512 o'lchamli embedding'lari (InsightFace `buffalo_sc`) cosine o'xshashlik bilan solishtiriladi. Davomat ketma-ket 3 kadrda moslik hamda ko'z pirpiratish va harakat orqali liveness tekshiruvidan o'tgandan keyingina yoziladi.
- **Yo'l ko'rsatuvchi enrollment wizard** — iPhone Face ID uslubida markaz, chap, o'ng, yuqori va past — 5 ta holatni ko'rsatib beradi, har bir holat barqaror ushlab turilsa avtomatik suratga oladi.
- **Offline-first dizayn** — Avval SQLite'ga (WAL rejimi) yoziladi, keyin fonda Google Sheets bilan sinxronlanadi; tarmoq uzilsa ham davomat yozuvlari yo'qolmaydi.
- **Galaxy Tab'da on-device topshirish** — O'lchov natijalariga tayanib yakuniy platforma sifatida Samsung Galaxy Tab'ni tanladim va Kotlin, CameraX, TFLite yordamida inferensni planshetning o'zida ishlaydigan qilib ko'chirdim. Joyga faqat bitta planshet o'rnatiladi.
- **Boshqaruv vositalari** — Flask veb-ilova va administrator dashboard'ini qo'shdim, testlarni GitHub Actions bilan avtomatlashtirdim.

Yuzni tanib davomatni qayd etuvchi kiosk

Yuzni tanish pipeline'i · benchmark natijalari



Python vs Hailo-8 NPU

Raspberry Pi 5 · bir xil rasm · har bir yuz uchun detection+embedding (oxirgi ustun: 5 ta yuzli bitta kadr)

Pipeline	Mediana latency	Jitter	O'tkazuvchanlik	5 yuzli kadr
Python + CPU (4 threads)	48.08 ms	±27 ms	~21 fps	124.21 ms
Hailo-8 NPU	**5.59 ms**	**±0.29 ms**	**~180 fps**	**20.37 ms**

> Latency'ning 8.6 baravar qisqarishidan ko'ra **jitter'ning 95 baravar kamaygani** amalda hal qiluvchi bo'ldi. O'rtacha tez bo'lsa ham, tarqoqlik katta bo'lsa kiosk sekin tuyuladi.

Python vs C++ port

Raspberry Pi 4 · bitta detection+recognition (yuqoridagi Pi 5 natijalaridan alohida o'lchangan)

Implementatsiya	Bitta detection+recognition	Sikl overhead'i
Python	~430 ms	~10 ms
C++ port	~410 ms	~1 ms

> C++ port faqat sikl overhead'ini kamaytirdi, umumiy vaqtni esa model hisoblashlari belgiladi. Tilni almashtirishning o'zi yetmadi — natijani NPU o'zgartirdi.

Natijalar

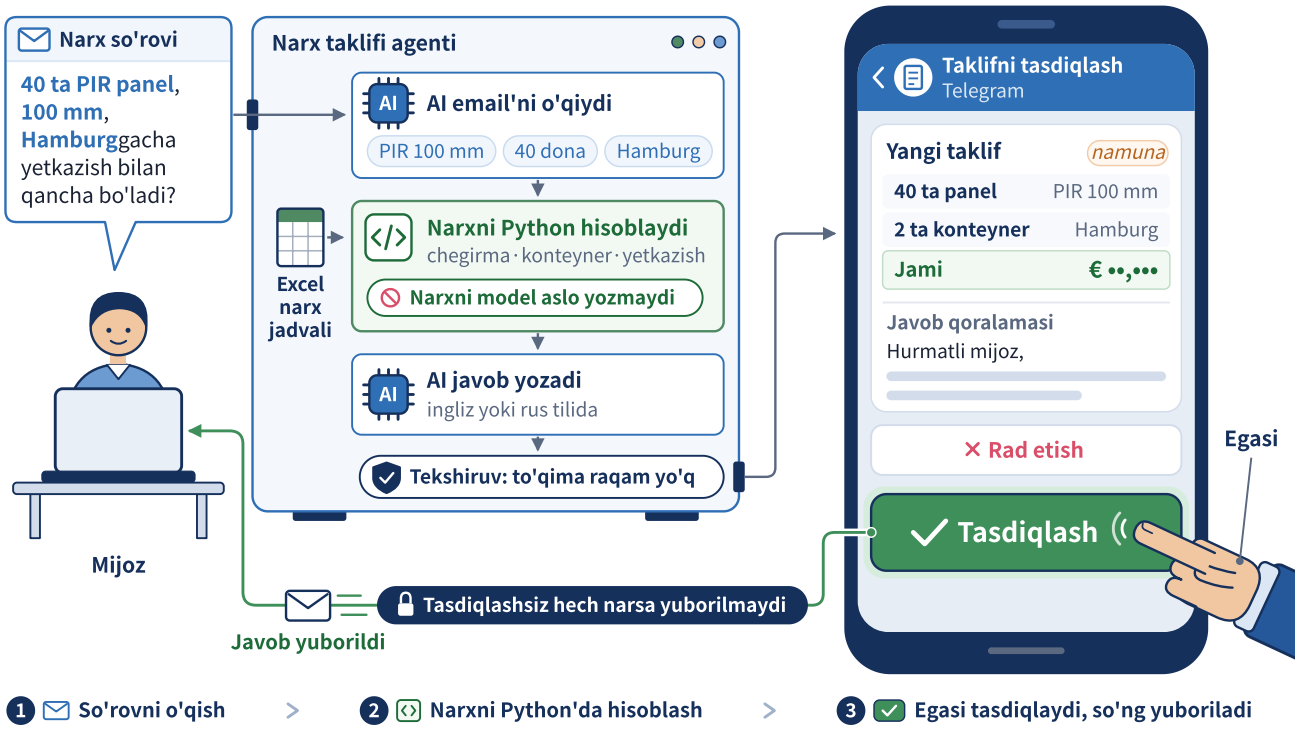
8.6×	95×	3 xil	Galaxy Tab
NPU inferens tezlashuvi	Jitter kamaydi	Pipeline o'lchandi	On-device topshirildi

06 Sales Agent

Email orqali narx taklifi beruvchi agent: narxni Python hisoblaydi, javobni Claude yozadi, egasi tasdiqlaydi

Frilans shartnoma loyihasi · AI/ML muhandisi · 2026.10 · **github.com/Nodir0705/sales-quote-agent**

Python | Claude API | Structured Outputs | Prompt Caching | Batch API | Telegram Bot API | SQLite



Biznes ta'siri

Agent narx so'rovining takrorlanadigan qismini bajaradi: xatni o'qiydi, kompaniyaning narx jadvalidan narx taklifini hisoblaydi va javob yozadi. Kompaniya egasi Telegram'dagi kartani o'qib, Tasdiqlash yoki Rad etish tugmasini bosadi; shu tugma bosilmaguncha mijozga hech narsa yetib bormaydi. Agentni qurib, haqiqiy pochta qutisi va Telegram bilan test rejimida ishga tushirib ko'rdim.

Muammo

Mijoz — izolyatsiyali sendvich panellar eksport qiluvchi kompaniya. Uning savdo pochtasiga keladigan xatlarning aksariyati bir xil turdagi so'rov xatlari edi: qandaydir panellarni ma'lum joyga yetkazib berish narxi so'ralardi. Har bir xat uchun Excel narx jadvalini ochish, chegirma darajasini qo'llash, buyurtma nechta konteynerni to'ldirishini hisoblash, yuk tashish narxini qo'shish va ingliz yoki rus tilida xushmuomala javob yozish kerak edi. Agentni ikki qoida asosida qurdim: narxlar modeldan emas, narx jadvalidan olinadi va har bir javobni yuborilishidan oldin egasi ko'radi.

Sales Agent

Texnik yondashuv

- **AI chaqiruvidan oldin filtrlar** — Bounce, avto-javob va newsletter xatlari, pochtaning o'z xatlari, ignore ro'yxatidagi va o'z limitidan oshgan jo'natuvchilar pullik chaqiruvgacha chetlatiladi.
- **AI xatni qat'iy sxema bo'yicha o'qiydi** — Birinchi Claude chaqiruvi structured output qaytaradi: mahsulotlar, miqdorlar, manzil, mijozning savollari va til. Xat modelga ishonchsiz ma'lumot deb belgilangan holda beriladi.
- **Narxni Python hisoblaydi** — Mahsulot va manzil Excel narx jadvalidan topiladi, aniq decimal arifmetika bilan chegirma darajasi qo'llanadi, konteynerlar sanaladi, yuk tashish narxi qo'shiladi. Mahsulot noaniq yoki manzil noma'lum bo'lsa, narx taxmin qilinmaydi, aniqlashtirish so'raladi.
- **Model hech qachon narx yozmaydi** — Ikkinchi chaqiruv javobni placeholder atrofida yozadi, Python esa o'zining narx blokini qo'yadi. So'ng Python tekshiruvlari qoralamani ko'rib chiqadi: Python bermagan pul summasi yoki muddat chiqsa, yuborish bloklanadi.
- **Tasdiqlashsiz hech narsa yuborilmaydi** — Egasi Telegram'da hisob-kitob, tekshiruvlar va javobning aniq matni bor kartani oladi. Tasdiq bazada bir marta qayd etiladi: ikki marta bosish yoki qayta ishga tushish xatni ikki marta yubora olmaydi, qabul qiluvchi esa qoralama yaratilganda belgilanadi.
- **Chaqiruvlar narxlanadi, freymvork yo'q** — Har bir AI chaqiruvi narxi bilan saqlanadi, kunlik xarajat chegarasi va jo'natuvchi limiti bor; prompt'lar keshlanadi, tunda kelgan so'rovlar Batch API orqali o'tadi. Agent freymvorki yo'q: bitta Python sikli va bitta SQLite fayl, keyingi dasturchi uni oson o'qiydi.

Natijalar

26/26	26/26	1 513 ta	Tasdiqlash
Extraction eval	Javob yozish tekshiruvi	Avtomatik tekshiruvlar	Har bir javob uchun shart
Barcha maydonlar to'g'ri	Pullik API'da	Hammasi o'tdi	Egasining Telegram kartasi

Yozma test xatlari, mijoz xatlari emas · Holati: qurilgan, haqiqiy pochta qutisi va Telegram bilan test rejimida sinalgan

Test	Nimani qamraydi	Natija
Extraction eval	Ingliz va rus tilida yozilgan xatlar: so'rovlar, follow-up'lar, savollar va aloqasiz xatlar; har bir maydon to'g'ri bo'lishi shart	26/26
Javob yozish testi	Haqiqiy (pullik) Claude API chaqiruvlari ingliz va rus tilidagi test xatlariga, jumladan prompt-injection xatiga javob yozadi	26/26 tekshiruv
Avtomatik tekshiruvlar	Narx hisobi, baza, javob tekshiruvlari va Telegram; barcha tashqi servislar fake bilan almashtirilgan; GitHub Actions'da ishlaydi	1 513 ta o'tdi

Cheklovlar: tekshiruvlar isbot emas, pattern tekshiruvlari (raqamsiz muddatni, masalan, “bir necha hafta”ni o'tkazib yuboradi), shuning uchun har bir javobni inson tasdiqlaydi. Javoblar faqat ingliz yoki rus tilida. Extraction eval haqiqiy mijoz xatlarida emas, test uchun yozilgan xatlarda o'tkazilgan.

07

Rasm asosidagi klassifikator barqarorligi tadqiqoti

Klassifikatorni qurganimdan keyin uning aniqligiga ishonsa bo'ladimi — buni tekshirishga ko'proq vaqt ketdi

Shaxsiy loyiha · MalImg dataseti (25 ta oila · ~9 000 namuna) · 2026 ·
github.com/Nodir0705/malware_classifier

ResNet50 | PyTorch | PGD | Adversarial Robustness | Grad-CAM | MalConv | FastAPI | Docker

> **Biznes ta'siri**
> Modelni bitta aniqlik ko'rsatkichiga qarab baholab bo'lmasligini raqamlar bilan ko'rsatgan loyiha. Production modellarni tekshirishda nimalarga qarash kerakligi haqidagi o'z mezonlarim shu yerda shakllandi.

Muammo

Malware binary fayllarini bayt darajasida rasmga aylantirib, oilasini tasniflaydigan model qurdim. Test to'plamidagi aniqlik 98.95% bo'ldi. Keyingi savol shu edi — bu raqamga haqiqatan ishonsa bo'ladimi?

Texnik yondashuv

- **Asosiy tuzilma** — Binary fayldagi har bir baytni bitta kulrang (grayscale) pikselga aylantiruvchi modulni o'zim yozdim (ochiq dataset faqat rasmlarni beradi) va ResNet50 ustiga 25 sinfli klassifikatsiya head'ini qo'ydim.
- **Sinf vaznli loss** — Oilalardagi namunalar soni ~3 000 tadan 80 tagacha notekis edi. Vaznlarni qo'llaganimda `Autorun.K` oilasining **F1 ko'rsatkichi 0 → 1.0** ga o'zgardi, umumiy aniqlik esa deyarli o'zgarmadi. Faqat umumiy aniqlikka qaraganimda bu muammoni ko'rmay qolardim.
- **Transfer learning tahlili** — Fine-tuning davomida backbone vaznlari **1% dan kam** (nisbiy L2) o'zgardi. Yaxshilanish amalda yangi klassifikatsiya head'idan keldi, ya'ni ImageNet belgilari byteplot'larga deyarli o'zgarishsiz o'tadi.
- **Barqarorlikni tekshirish** — Alohida data split va alohida bazaviy model bilan takrorlanadigan PGD hujumi va adversarial qayta o'qitish tajribalarini qurdim, ustiga yozish (overwrite) hujumi bilan 3 xil arxitekturani taqqosladim. Natijalar keyingi sahifada.
- **Interpretatsiya va deployment** — Grad-CAM bilan model faylning qaysi qismiga qarashini tekshirdim, klassifikatorni FastAPI veb-ilova, CLI va Docker ko'rinishida paketladim.

Natijalar

98.95%	22% → 94.15%	3 xil	Grad-CAM
Klassifikatsiya aniqligi	PGD hujumidagi aniqlik	Arxitektura taqqoslash	Interpretatsiyani tekshirish
Deploy modeli · test to'plami	Alohida bazaviy model	Aniqlik vs barqarorlik	Model qaragan qismni ko'rish

Rasm asosidagi klassifikator barqarorligi tadqiqoti

Barqarorlikni tekshirish natijalari

Adversarial hujumlarga qarshi tekshiruv — takrorlanadigan tajriba

Deploy qilingan klassifikatordan (98.95%) alohida, repoda takrorlanadigan tajriba qurdim. Data split boshqa (test to'plami 957 ta), bazaviy model alohida o'qitilgan ResNet50 (toza aniqlik 99.48%). Har bayt ko'pi bilan 2 ga o'zgaradigan PGD-20 (ε=2/255) aniqlikni 22.15% ga, 4 da (ε=4/255) 6.58% ga tushirdi.

Model	Toza kirish	PGD-20 ε=2/255	PGD-20 ε=4/255
Bazaviy model (fine-tuned ResNet50)	99.48%	22.15%	6.58%
Adversarial training (butun tarmoq, PGD ε=2/255)	**97.18%**	**94.15%**	**65.20%**

> Butun tarmoqni PGD (ε=2/255) namunalari bilan qayta o'qitganimda hujum ostidagi aniqlik **94.15%** ga tiklandi, evaziga toza aniqlik **2.3 punkt** pasaydi. Ammo o'qitishdagidan ikki baravar kuchli hujumda (ε=4/255) aniqlik 65.20% bo'ldi — barqarorlik kuchliroq hujumlarga to'liq o'tmadi.

Arxitekturalar bo'yicha aniqlik va barqarorlik trade-off'i

Model	Aniqlik	Faylning 50% ustiga yozilganda qolgan aniqlik
ResNet50 (tasvir asosida)	98.95%	43.50%
MalConv (bayt 1D CNN)	94.03%	64.09%
Byte-level Transformer	96.06%	18.27%

> Eng aniq model eng barqaror bo'lib chiqmadi. MalConv aniqligi eng past, lekin ustiga yozish hujumiga eng yaxshi bardosh berdi — u butun faylga emas, bitta asosiy bayt patterniga tayanadi.

Kutilmagan natija — kam namunali yangi oilalarni aniqlash

Yangi malware oilasining bir nechtagina namunasi bo'lganda uni taniy olish-olmasligini tekshirish uchun metric learning qo'lladim. Natija **aksincha yomonlashdi**.

	O'qitilmagan ImageNet belgilari	O'zim o'qitgan versiya
Yangi oilalar aniqligi (5-way 5-shot)	98.22%	89.40%

> Sababi: o'qitish yaqin oilalarni belgilar fazosida bir-biriga siqib qo'ydi va mavjud oilaning yangi varianti qarindosh oilalar bilan qo'shilib ketdi. **Xulosa: yangi variantlarni aniqlashda o'qitilmagan ImageNet belgilari yaxshiroq.** Kutilgan natija emas edi, ammo bu tajribasiz noto'g'ri arxitekturani tanlagan bo'lardim.

Cheklov: yuqoridagi raqamlar MalImg datasetiga tegishli, u nisbatan eski Windows malware'laridan iborat. Zamonaviy malware'larga umumlashish tekshirilmagan.

08 IoT xavfsizlik shlyuzi

Bulutsiz, bitta platada: 4 qatlamli aniqlashdan avtomatik karantingacha

Shaxsiy loyiha · Orange Pi 5 (RK3588S) · 2026 · **github.com/Nodir0705/iot-gateway-security**

XDP/eBPF | Suricata | RK3588 NPU | RKNN | ONNX Runtime | Scikit-Learn | nftables | MQTT

> **Biznes ta'siri**
> Bulutsiz, bitta platada aniqlashdan karantingacha ishlashini 47 daqiqalik stress-testda tasdiqladim.

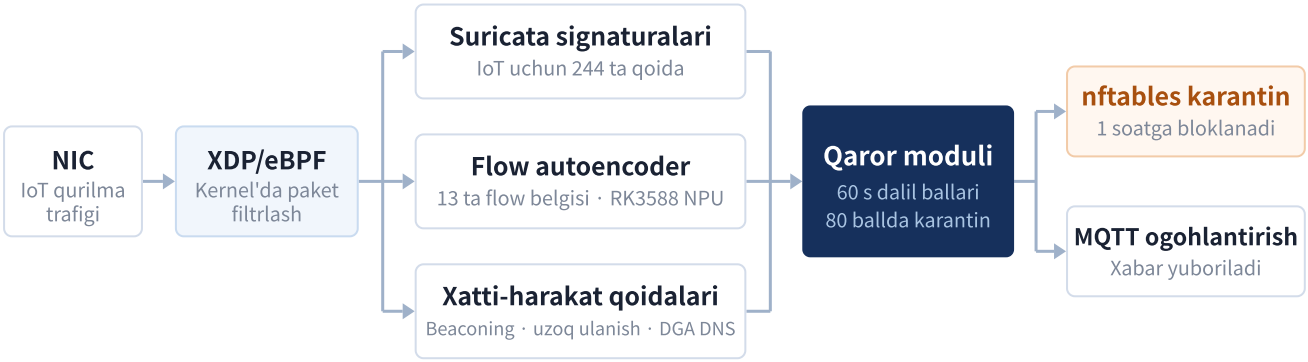
Muammo

IoT trafigini bulutsiz, bitta shlyuzda kuzatib, hujumda qurilmani avtomatik karantinga olish kerak edi. Ma'lum hujum va yangi anomaliyalar uchun turli qatlamlarni bitta qarorga birlashtirish lozim edi.

Texnik yondashuv

- **Kernel darajasida filtrlash** — XDP/eBPF NIC'ga kelgan paketlarni avval kernel darajasida filtrlaydi.
- **IoT uchun Suricata qoidalari** — IoT hujumlariga qaratilgan 244 ta Suricata qoidasini o'zim yozdim.
- **NPU'da flow autoencoder** — 13 ta flow belgisini qabul qiladigan autoencoder RK3588 NPU'da ishlaydi. Inferens RKNN (NPU) → ONNX (CPU) → scikit-learn random forest tartibida zaxiraga o'tadi.
- **Xatti-harakat qoidalari** — Beaconing, uzoq ulanishlar, DGA domenlarga DNS so'rovlari aniqlanadi.
- **Dalil ballari asosida karantin** — Qatlamlar natijalari ballga yig'iladi; 60 soniyada 80 ballga yetsa, qurilma nftables bilan 1 soatga avtomatik karantinga olinadi va MQTT orqali ogohlantirish yuboriladi.

Pipeline tuzilmasi



Natijalar · 47 daqiqalik stress-test, 9 bosqichli hujum

9/9	87/88 (98.9%)	244 ta	1 551 ta
Hujum bosqichi aniqlandi	Kutilgan qoida ishladi	IoT Suricata qoidalari	Ogohlantirish qayd etildi

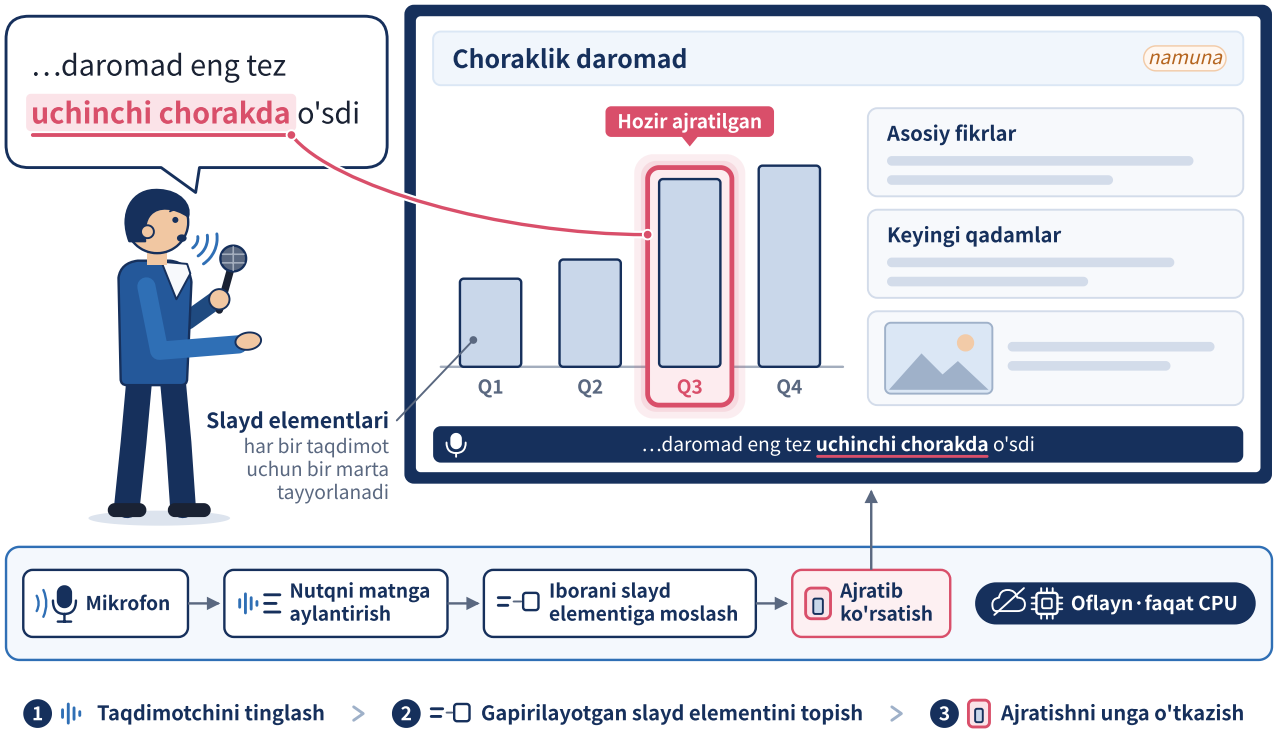
Cheklov: faqat laboratoriya uchun; modellar NSL-KDD va sintetik flow'larda o'qitilgan, real tarmoqda tekshirilmagan.

09 Smart Pointer

Taqdimotchi gapirayotgan qismni slaydda ajratib ko'rsatuvchi vosita

Shaxsiy loyiha · 2026 · **github.com/Nodir0705/smart-pointer**

Python | sherpa-onnx | Canary 180M Flash | TinyBERT Cross-Encoder | python-pptx | Claude API



Biznes ta'siri

Taqdimotchi gapirayotganda slaydning aynan u gapirayotgan qismi ajratib ko'rsatiladi, shuning uchun tinglovchilar har bir gap nimaga tegishli ekanini ko'radi. Jonli sikl CPU'da oflayn ishlaydi; tarmoq faqat bir marta, taqdimot tayyorlanayotganda kerak bo'ladi.

Muammo

Taqdimotchi bir vaqtda slaydning bitta qismi haqida gapiradi. Men har bir gapni eshitib, u qaysi blok, karta, diagramma ustuni yoki jadval katagi haqida ekanini aniqlaydigan va nutq davom etar ekan, ajratishni o'sha yerga ko'chiradigan vosita qurmoqchi edim. Taqdimotchi slayd haqida gapirmayotganda esa vosita hech narsani ajratmasligi kerak.

Smart Pointer

Texnik yondashuv

- **Har bir taqdimot bir marta tayyorlanadi** — .pptx fayl python-pptx va slaydlarning xom XML'i orqali tahlil qilinib, render qilinadi; matn bloklari aynan PDF qatlamidan olinadi, rasmlardagi matnni OCR o'qiydi. So'ng model slayd xaritasini yozadi: taqdimotchi ko'rsatishi mumkin bo'lgan elementlar va har biri haqida aytilishi mumkin bo'lgan qisqa iboralar. Xaritalarni Claude API yozadi (zaxira varianti lokal Qwen modeli), shuning uchun tarmoq faqat bir marta, tayyorlash bosqichida kerak.
- **CPU'dagi jonli sikl** — Mikrofon → nutqni matnga aylantirish (Canary 180M Flash) → iboralar → 2 qatlamli TinyBERT cross-encoder har bir iborani har bir slayd elementi bilan baholaydi → ko'rsatish va ushlab turish → brauzerda ajratib ko'rsatish.
- **Ko'rsatish va ushlab turish** — Pointer har bir so'zda emas, har bir iborada bir marta qaror qiladi. O'zgarmas “hech biri” chegarasi taqdimotchi slayd haqida gapirmayotganda hech narsani yoritmaydi, tanlangan element esa yangi ibora boshqa joyni aniq ko'rsatmaguncha saqlanadi.
- **Deterministik** — Bitta CPU thread'i, qat'iy belgilangan model versiyalari va jonli siklda tarmoq yo'q: bir xil audio doim bir xil ajratishlarni beradi, shuning uchun har bir o'zgarishni yozib olingan nutqlarda aniq taqqoslash mumkin.
- **Baholash stendi** — Nutq ssenariylari slaydlarni tartib bilan o'qiydi, u yerdan bu yerga sakraydi yoki narsalarni ma'nosi orqali tilga oladi; har bir gapga u tegishli element label qilib qo'yilgan. Ularni sintetik ovoz (TTS) o'qiydi, audio esa nutqni matnga aylantirish va pointer orqali real vaqtdan tezroq o'tkaziladi.
- **Held-out taqdimotlar** — 7 ta ochiq taqdimotni yuklab olib, testdan oldin muzlatib qo'ydim. Ishlab chiqish davomida ularning slaydlarini hech kim ochmadi, ular yakuniy sozlamalar bilan, qo'shimcha sozlashsiz baholandi.

Natijalar

77.3%	2.4	16 ms	2 078 ta
Yoritilgan ishora qadami	Nutq boshiga xato ajratish	Har bir re-ranker qarori	Baholangan ishora qadami
Sintetik ovoz · 7 taqdimot	Sintetik ovoz · 21 nutq	Bitta CPU thread'i	Ko'rilmagan 7 taqdimotda

Pointer natijalari yozma ssenariylarni o'qiyotgan sintetik ovoz (TTS) bilan olingan. Haqiqiy taqdimotda hali o'lchanmagan.

Nutqni matnga aylantirish: haqiqiy ovozdagi so'z xatosi darajasi bo'yicha tanlandi	So'z xatosi darajasi
Canary 180M Flash (tanlandi)	**11.3%**
Whisper large-v3-turbo	32.1%

Cheklovlar: barcha pointer natijalari sintetik ovoz (TTS) bilan olingan, haqiqiy taqdimotda hali o'lchanmagan. Faqat ingliz tilida ishlaydi. Slayd xaritalari uchun taqdimot tayyorlanayotganda bir marta tarmoq kerak; oflayn faqat jonli sikl ishlaydi.

Texnik ko'nikmalar

Computer Vision	Object Detection, Semantic / Instance Segmentation, OCR, Multi-Object Tracking, Face Recognition (ArcFace), Pose Estimation
ML / DL freymvorklar	PyTorch, TensorFlow, Keras, Scikit-Learn, Ultralytics (YOLO), OpenCV, InsightFace, Hugging Face Transformers, SAM2
Optimallashtirish	Quantization, Pruning, TensorRT, ONNX Runtime, DLA offloading, backbone almashtirish va A/B baholash
Edge qurilmalar	NVIDIA Jetson (Orin NX · Orin Nano), Hailo-8 NPU, RK3588 NPU (Orange Pi 5), Raspberry Pi 4/5, Android planshetlar
Deployment	Docker, FastAPI, Flask, REST API, PostgreSQL, SQLite, parallel multi-stream serving
Baholash va monitoring	Sinflar bo'yicha metrikalar, adversarial robustness testlari, Grad-CAM, FPS/latency/jitter benchmarking, W&B, MLflow, TensorBoard
LLM va agentlar	Claude API (structured outputs, prompt caching, Batch API), LLM'larni baholash, nutqni matnga aylantirish (sherpa-onnx), cross-encoder re-ranking
Tillar va vositalar	Python (asosiy), C++, SQL, Linux (Ubuntu), Git/GitHub, GitLab, GitHub Actions CI

Ta'lim · Tillar · Viza

Ta'lim

Kompyuter fanlari bakalavri	Woosong universiteti · Daejeon · 2019 – 2023

Viza

E-7 vizasi	Special Ability · ishlash huquqi bor

Tillar

O'zbek tili	Ona tili
Ingliz tili	Erkin (Fluent)
Koreys tili	Kundalik muloqot darajasida
Rus tili	Kundalik muloqot darajasida

Mentor · Woosong universiteti capstone loyihasi · 2024

Computer vision jamoasiga (tasvirga ishlov berish · YOLO modelini o'qitish) har hafta fikr-mulohaza berib yo'l-yo'riq ko'rsatdim; jamoaning 200 dan ortiq jamoa orasida 1-o'rinni olishiga hissa qo'shdim.

Nodirbek Makhtumov

AI/ML muhandisi · Computer Vision · Edge AI · Modellarni optimallashtirish

stark.developer2877@gmail.com · +82 10-5751-0750
github.com/Nodir0705 · linkedin.com/in/nodirbek-makhtumov-03084a194
nodir0705.github.io